EDyGS: Event Enhanced Dynamic 3D Radiance Fields from Blurry Monocular Video (Supplementary Material)

7 Overview

In this supplemental material, we present: (1) further details of our approach, including implementation details, in Section 8; (2) preliminary of 3D Gaussian Splatting and Event Camera in Section 9; and (3) ablation study in Section 11; (4) additional experimental results in Section 12, which shows more quantitative and qualitative comparisons between our method and other approaches; and (5) details about the differential path in Section 10.

8 Other Details of Our Method

Implementation Details. Our implementation is built using the PyTorch framework. We conducted experiments on a single RTX 3090 GPU with approximately one and a half hours to optimize a scene. The learning rate, optimizer, and training iterations are consistent with Deformable 3DGS [Yang *et al.*, 2024]. In the initial 3000 iterations, we jointly train the static 3DGS and pose. The learning rate of pose for rotation and translation is 1e-5 and 1e-4, respectively. After 3000 epochs, we optimize the deformation field, the 3D Gaussians, and fine-tune the pose. The learning rate of pose for rotation becomes 1e-6. We set the number of latent sharp images k to 5, with an event threshold of 0.2 for synthetic scenes and real scenes.

9 Preliminary

3D Gaussian Splatting. 3D Gaussian Splatting (3DGS) [Kerbl *et al.*, 2023] offers an explicit method for representing a dynamic 3D scene, modeling the appearance, geometry, and scene dynamics with a set of Gaussians. Each 3D Gaussian is defined by the center point $\mu \in R^3$ and a 3D covariance matrix $\Sigma \in R^{3\times3}$.

$$G(x) = \exp\left(-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)\right), \tag{9}$$

whereΣ is decomposed and defined by the matrix S and R as:

$$\Sigma = \mathbf{RSS}^T\mathbf{R}^T. \tag{10}$$

The matrix $\mathbf{S}$ is constructed from the 3D-vector scaling factor $\mathbf{s} \in R^3$. The matrix $\mathbf{R}$ is constructed from the rotation quaternion $\mathbf{r} \in R^4$. The 3D Gaussians are projected to 2D image space [Zwicker *et al.*, 2001] to synthesize novel views. According to the Jacobian matrix $\mathbf{J}$ of the affine approximation of the projective transformation and the viewing transformation $\mathbf{W}$, the covariance matrix Σ' in camera coordinates is expressed as:

$$\Sigma' = \mathbf{JW}\Sigma\mathbf{W}^T\mathbf{J}^T. \tag{11}$$

The final color of each pixel is rendered by the α-blending:

$$C = \sum_{i\in\mathcal{N}} c_i\alpha_i\prod_{j=1}^{i-1}(1-\alpha_j). \tag{12}$$

The value of c_i is the color of each point, determined by the view direction and spherical harmonics. α_i is obtained by multiplying the transformed 2D Gaussian with opacity.

Event Camera. At timestamp τ, when the brightness variation of the pixel (x, y) in the logarithmic domain exceeds the threshold Θ, the event camera records an event $e(x, y, \tau, p)$. Here, p represents the polarity, defined as:

$$p_{x,y,\tau} = \begin{cases} -1, & \log(I_{x,y,\tau}) - \log(I_{x,y,\tau-\Delta\tau}) < -\Theta, \\ +1, & \log(I_{x,y,\tau}) - \log(I_{x,y,\tau-\Delta\tau}) > \Theta. \end{cases} \tag{13}$$

To align with the image frames, we divide and accumulate the events within the exposure time of a blurry image :

$$\{\mathbf{e}_k\}_{k=0}^{n-1} = \{(x_i, y_i, \tau_i, p_i) \mid t_k < \tau_i \le t_{k+1}\}, \tag{14}$$

where t_k, t_{k+1} are the start timestamp and end timestamp of the exposure period for a blurry image.

10 Details about the Differential Path

For this path:

$$\begin{aligned} \frac{\partial\hat{C}_{\text{st}}^i}{\partial P_i} &= \frac{\partial\hat{C}_i}{\partial P_i} - M_i\frac{\partial\hat{C}_i}{\partial P_i} - \hat{C}_i\frac{\partial M_i}{\partial P_i} \\ &= (1-M_i)\frac{\partial\hat{C}_i}{\partial P_i} - \hat{C}_i\sum_{t=1}^{\mathcal{N}}\frac{\partial M_i}{\partial\alpha_t}\frac{\partial\alpha_t}{\partial P_i}, \end{aligned} \tag{15}$$

we differentiate $\frac{\partial M_i}{\partial\alpha_t}$ part:

$$\begin{aligned} &\tfrac{\partial M_i}{\partial\alpha_t} = \textstyle\sum_{t\in\mathcal{N}} m_t\prod_{u=1}^{t-1}(1-\alpha_u) - \frac{S_t}{1-\alpha_t}, \\ &\text{where } S_t = \sum_{u>t} m_u\alpha_u\prod_{v=1}^{u-1}(1-\alpha_v). \end{aligned} \tag{16}$$

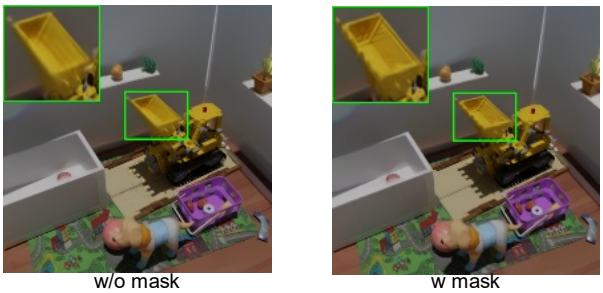

Figure 6: Ablation study on motion mask.

We can compute S_t as we iterate backward over Gaussians:

$$S_{\mathcal{N}} = 0, S_{n-1} = S_n + m_n \alpha_n \prod_{v=1}^{n-1}(1-\alpha_v). \tag{17}$$

We implement the backward pass of the gradient as a CUDA operation, following gsplat [Ye *et al.*, 2024].

We differentiate $\frac{\partial \hat{C}_i}{\partial P_i}$ and $\frac{\partial \alpha_t}{\partial P_i}$ part according to Cogs[Jiang *et al.*, 2024]:

$$\begin{aligned}
&\frac{\partial C_i}{\partial P_i} = \sum_{t\in\mathcal{N}} \frac{\partial C_i}{\partial \alpha_t}\frac{\partial \alpha_t}{\partial P_i}, \\
&\frac{\partial C_i}{\partial \alpha_t} = \sum_{t\in\mathcal{N}} c_t \prod_{u=1}^{t-1}(1-\alpha_u) - \frac{S_t}{1-\alpha_t}, \\
&\text{where } S_t = \sum_{u>t} c_u \alpha_u \prod_{v=1}^{u-1}(1-\alpha_v). \\
&\frac{\partial \alpha_t}{\partial P_i} = \frac{\partial \alpha_t}{\partial G'_t}\frac{\partial G'_t}{\partial \mu'_t}\frac{\partial \mu'_t}{\partial P_i} = \sigma_t G'_t \Sigma'^{-1}_t (\mathbf{x} - \mu'_t).
\end{aligned} \tag{18}$$

11 Other Ablation Study

Effect of motion mask. Although the numerical difference of 0.62 between using and not using a motion mask may not seem substantial, using a mask results in better reconstruction of the dynamic parts compared to not using a mask, as shown in Figure 6.

12 Quantitative and Qualitative Results

Tables 4 and 5 present the quantitative metrics of different methods for each scene, including deblurring results from input views and novel view synthesis. Figures 7, 8, 9, and 10 display the visual results of different methods for each scene. Each image demonstrates the overall visual result of different methods across various scenes, as well as the zoom-in of static and dynamic regions.

References

[Jiang *et al.*, 2024] Kaiwen Jiang, Yang Fu, Mukund Varma T, Yash Belhe, Xiaolong Wang, Hao Su, and Ravi Ramamoorthi. A construct-optimize approach to sparse view synthesis without camera pose. In *ACM SIGGRAPH 2024 Conference Papers*, 2024.

[Kerbl *et al.*, 2023] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.*, 2023.

[Yang *et al.*, 2024] Ziyi Yang, Xinyu Gao, Wen Zhou, Shaohui Jiao, Yuqing Zhang, and Xiaogang Jin. Deformable 3d gaussians for high-fidelity monocular dynamic scene reconstruction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.

[Ye *et al.*, 2024] Vickie Ye, Ruilong Li, Justin Kerr, Matias Turkulainen, Brent Yi, Zhuoyang Pan, Otto Seiskari, Jianbo Ye, Jeffrey Hu, Matthew Tancik, and Angjoo Kanazawa. gsplat: An open-source library for gaussian splatting, 2024.

[Zwicker *et al.*, 2001] Matthias Zwicker, Hanspeter Pfister, Jeroen Van Baar, and Markus Gross. Ewa volume splatting. In *Proceedings Visualization, 2001. VIS'01.* IEEE, 2001.

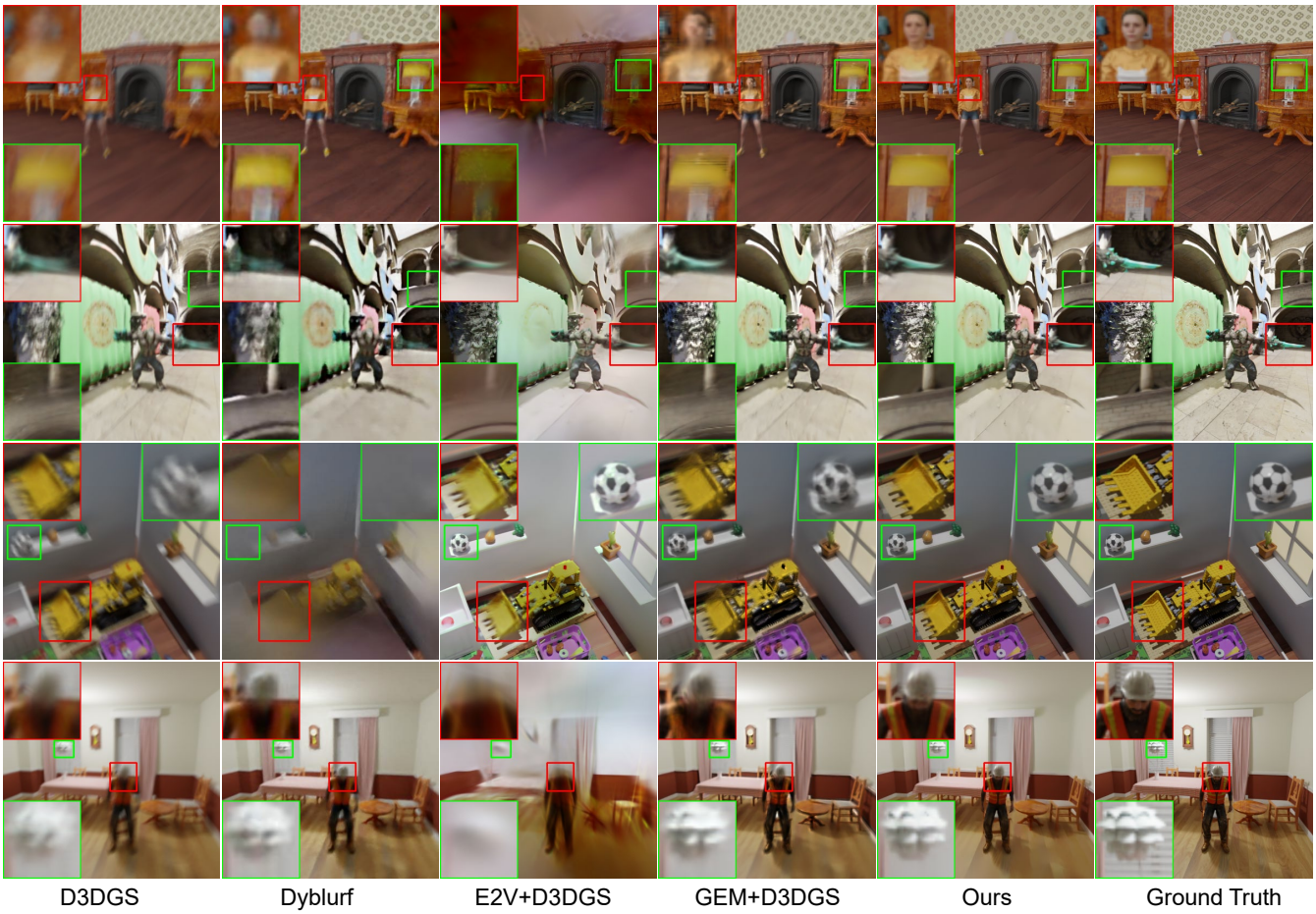

Figure 7: Input view synthesis results on our synthetic scenes. The red box highlights the detailed magnification of the dynamic region, while the green box shows the detailed magnification of the static region.

	D3DGS			Dyblurf			E2V+D3DGS			GEM+D3DGS			Ours		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
jump	26.85	.7079	.4664	27.39	.7395	.4161	18.15	.5067	.6213	28.49	.7980	.3674	30.94	.8506	.2775
mutant	23.14	.7861	.3214	21.96	.7531	.3523	20.18	.7795	.3161	24.93	.8446	.2597	29.90	.8990	.2018
room	24.81	.7807	.2946	22.95	.7343	.3354	22.74	.8690	.1610	27.26	.8752	.2010	33.12	.9538	.0881
standup	28.33	.8439	.2896	28.42	.8413	.2707	19.18	.7519	.4291	30.21	.8876	.2271	33.40	.9168	.2016
average	25.78	.7796	.3430	25.18	.7670	.3436	20.07	.7267	.3819	27.72	.8513	.2638	**31.84**	**.9050**	**.1923**

Table 4: Comparison of PSNR, SSIM, and LPIPS across different methods on deblurring view synthesis.

	D3DGS			Dyblurf			E2V+D3DGS			GEM+D3DGS			Ours		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
jump	26.55	.7067	.4675	23.20	.5552	.4761	20.65	.5740	.5587	28.01	.7955	.3671	30.38	.8491	.2795
mutant	20.94	.7333	.3442	17.98	.6132	.4051	19.41	.7521	.3280	22.14	.7937	.2814	27.98	.8849	.2061
room	24.74	.7793	.2950	18.71	.6074	.3999	22.62	.8618	.1642	27.09	.8724	.2022	32.18	.9487	.0910
standup	28.30	.8433	.2901	21.24	.6511	.3873	19.12	.7485	.4317	30.19	.8866	.2280	33.04	.9146	.2033
average	25.13	.7657	.3269	20.28	.6067	.4171	20.45	.7341	.3707	26.86	.8370	.2697	**30.90**	**.8993**	**.1950**

Table 5: Comparison of PSNR, SSIM, and LPIPS across different methods on novel view synthesis.

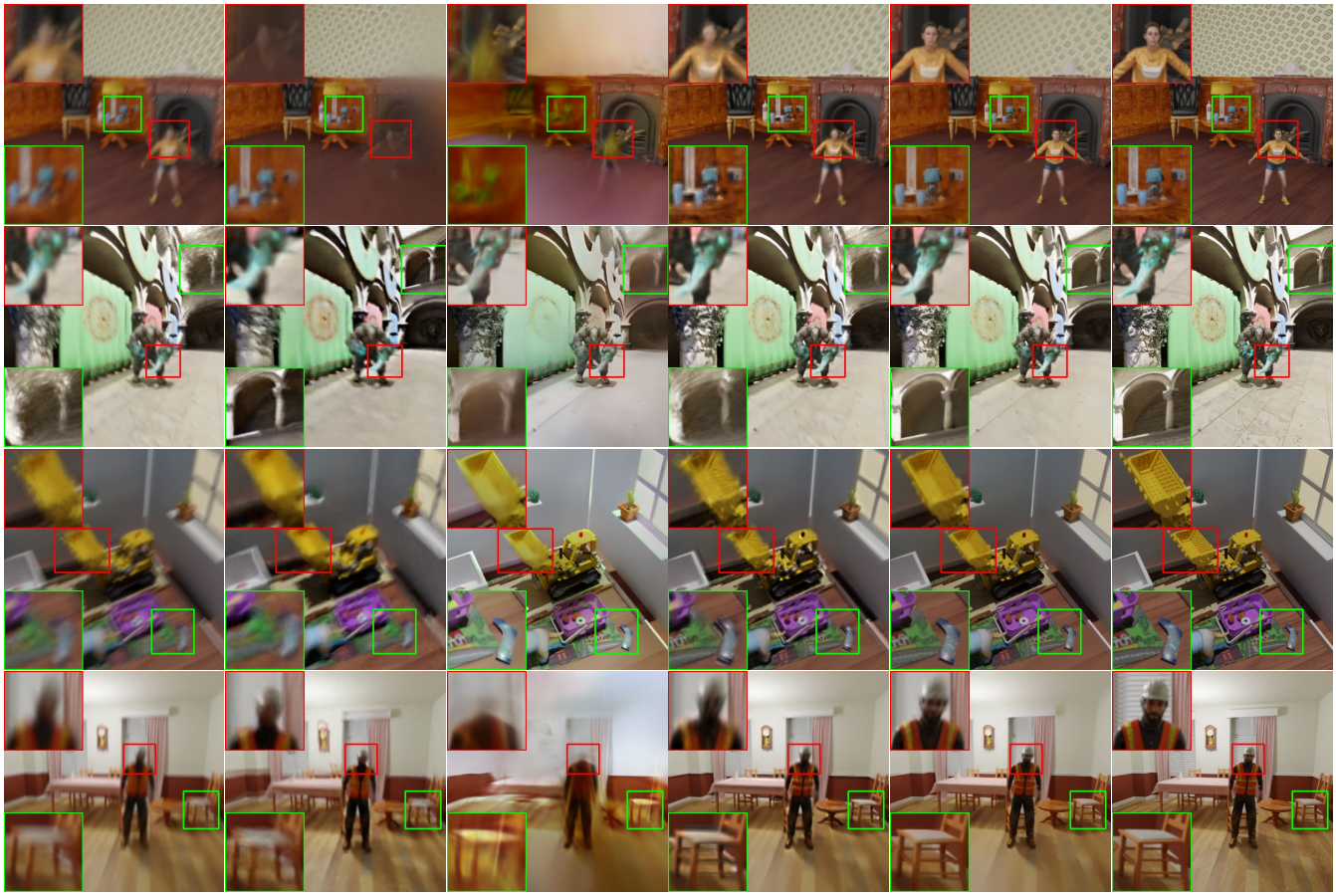

Figure 8: Novel view synthesis results on our synthetic scenes. The red box highlights the detailed magnification of the dynamic region, while the green box shows the detailed magnification of the static region.

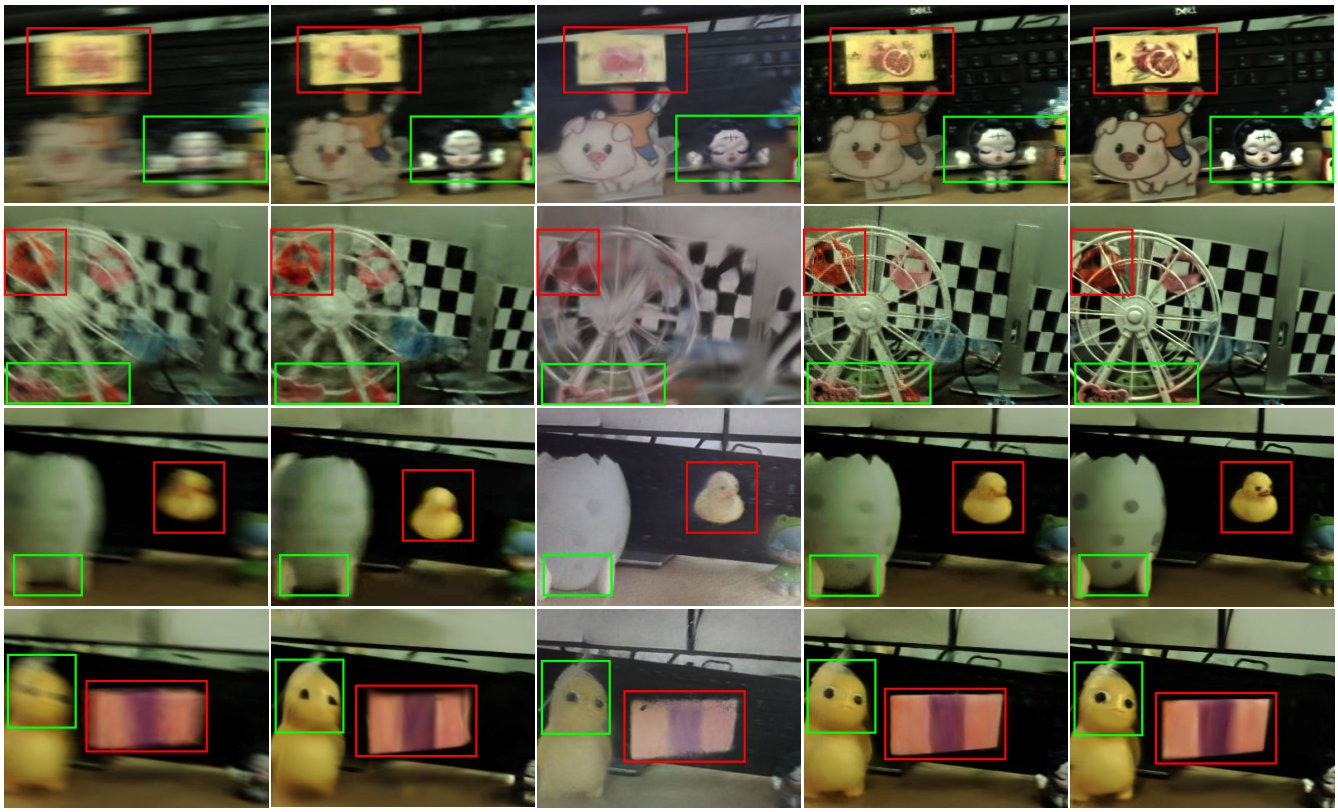

Figure 9: Input view synthesis results on real-world scenes. The red box highlights the detailed magnification of the dynamic region, while the green box shows the detailed magnification of the static region.

Figure 10: Novel view synthesis results on real-world scenes. The red box highlights the detailed magnification of the dynamic region, while the green box shows the detailed magnification of the static region.